

线程在创建完工作线程后，自身也作为一个工作线程承担一部分计算任务，这样一共 8 个线程)，线程数量并不是越多越好。

2 ANN 选题：多线程实验

2.1 在 SIMD 实验基础上初步探索多线程优化

2.1.1 Flat-SIMD + 多线程并行 (2 分)

在 SIMD 实验中 Flat-SIMD 的基础上，进一步使用 Pthread/OpenMP 进行并行优化，可以考虑 query 级并行或者 base 向量划分并行（此时需要考虑 topk 的 reduce，因此每个线程求出的 top p 中 p 的取值也会影响 recall 和 latency 的 trade-off）。此外，也可以尝试通过调整 schedule 策略等方式实现一定程度上的负载均衡。

2.1.2 PQ-SIMD + 多线程优化 (2 分)

PQ-SIMD 分成两个阶段。

第一阶段是 PQ 编码阶段，主要是 KMeans/KMeans++ 聚类与向量编码。如果在 SIMD 实验中已经实现了相关优化，也可以继续在此基础上进行多线程优化。这一部分不作为性能考核重点，但如果实现了较高质量的并行优化，可以作为进阶要求加分。

第二阶段是查询阶段（默认使用 ADC），又可以分成两个子阶段。

(1) LUT 构建阶段

LUT 构建阶段本质上仍然属于 dense compute。每个子空间中，需要分别计算每个子查询向量到所有类中心的距离，因此可以考虑：

- 子空间并行；
- 子查询向量并行；
- 类中心集合并行；

如果 SIMD 实验中已经实现了跨 centroid 并行，那么这里也可以进一步结合 Pthread/OpenMP 进行优化。这一部分整体计算较规则，通常能够获得比较稳定的并行加速效果。

(2) 查表累加阶段

查表累加阶段从“计算距离”转变为“查表 + 累加”，瓶颈逐渐从 compute-bound 转向 memory-bound。由于 PQ code 对 LUT 的访问通常是随机的，因此 cache miss 与内存带宽会逐渐成为主要限制因素。

在这一阶段，推荐探索 batch query 并行，即不同线程分别处理不同 query；或者，对于单个 query 的查表扫描，也可以尝试对 base 向量集合进行分块并行扫描，并由每个线程维护局部 top-p，最后再进行 merge；但由于这一阶段通常已经受到 memory bandwidth 限制，因此线程数并不一定是越多越好，可能导致负优化，在报告中分析清楚就可以。

如果 SIMD 中实现了 gather、shuffle (FastScan)、批量 gather 等进阶方法，进一步结合这些方法探索与多线程结合下的性能优化。

2.2 IVF (Inverted File)

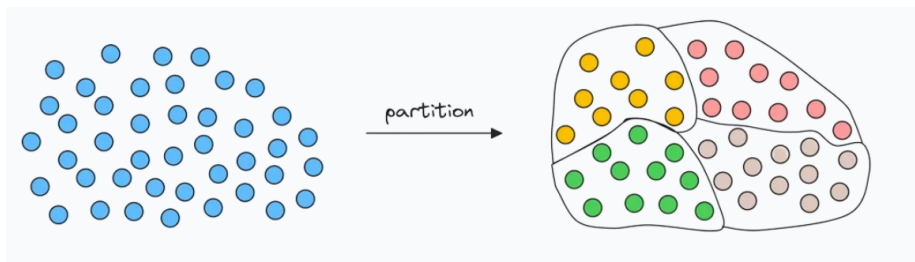


图 2.1: IVF 索引构建

IVF 的索引构建如图2.1所示，只需要使用任意一种聚类算法对 base data 进行聚类，并保存每个簇的质心和该簇中点的编号集合（簇的个数通常根据 base data 的大小决定，一般设置为 64 到 4096）。在 IVF 的索引构建中，还可以对内存进行重排，将相同簇的向量存储在一起，降低查询时的 cache miss 率。

这一部分的性能不作为实验重点要求，如果有兴趣，也可以仿照 PQ 编码阶段的优化方法进一步探索 SIMD 与多线程优化。

IVF 的查询过程步骤一如图2.2和2.3所示，首先计算查询点到每个簇的距离，只需要计算这些簇中点到查询的距离并进行排序，选出距离查询点最近的前 nprobe 个簇，其中 nprobe 为搜索时的参数，更高的 nprobe 意味着更好的准确度，但也会导致延迟更高，实验结果中需要调整 nprobe 的值来绘制算法的 recall-latency 曲线。步骤二 rerank 是对选出来的 nprobe 个簇

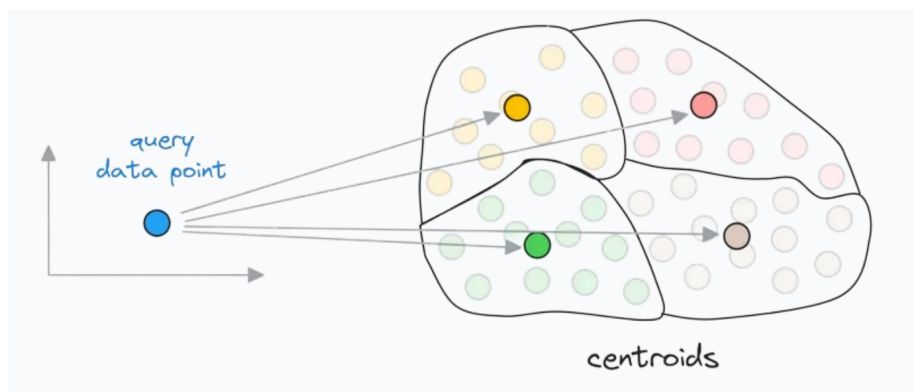


图 2.2: IVF 查询过程 I

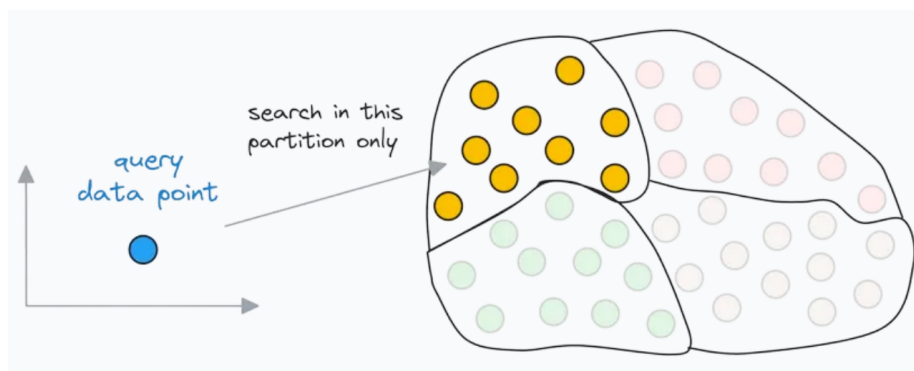


图 2.3: IVF 查询过程 II

中的点使用原始精度进行精确查询。

2.2.1 IVF-SIMD baseline (2 分) 及其多线程优化 (4 分)

在实现上，可以先实现 IVF-SIMD 作为 baseline，即使用 IVF 减少搜索点数量，并使用 SIMD 优化单次距离计算。在此基础上，可以进一步探索 Pthread/OpenMP 多线程优化。

IVF 查询阶段通常可以分成两个部分：

- 粗排：即计算 query 到所有簇中心的距离，并选出前 nprobe 个簇。这一阶段的计算量通常相对较小，因此即使进行多线程优化，也可能因为线程创建、同步与 reduce 开销导致负优化。同学们可以尝试 query

级并行、簇中心并行等方法, 并分析线程数与性能之间的关系。

- 精排: 即对选中的簇进行扫描与距离计算。这一阶段通常是 IVF 查询的主要开销来源, 因此更适合进行多线程优化。例如可以探索簇划分并行、query 级并行等等。

需要注意的是, 不同 inverted list 的长度通常并不均匀, 因此线程划分方式、schedule 策略等因素都会显著影响最终性能。有一些并行方法需要进行 topk reduce, 因此设置每一个线程的 top p 也会引起 trade-off。

2.2.2 IVF-PQ-SIMD baseline(3 分) 及其多线程优化 (5 分)

在实验中同样可以考虑将 IVF 和 PQ 结合, 一般有两种方法: (1) 先对所有 base data 进行 PQ, 再构建 IVF 索引。(2) 先构建 IVF 索引, 再在每个簇中分别进行 PQ。两者的准确性和性能会有一些差异, 同学们如果选择实现 IVF-PQ 算法的话可以尝试对该现象进行分析。

在 IVF-PQ 中, 由于 PQ 已经显著降低了单次距离计算量, 因此系统瓶颈会进一步从 compute-bound 转向 memory-bound。在这一部分继续进行多线程优化时, 并不一定线程数越多越快, 甚至可能出现负优化现象, 因此需要结合 latency、recall 与具体实现和线程数综合分析系统性能。

2.3 HNSW (Hierarchical Navigable Small World graphs) 进阶要求, 2 分

对于 HNSW 图索引的并行实现, 同学们可以直接在 hnswlib 的基础上进行开发, hnswlib 已经整理到了服务器的框架中并提供了索引构建的示例, 这里便不再赘述。

对于图索引上的查询过程, 如图2.4所示, 首先从 HNSW 的最顶层开始, 在该层中找到距离查询最近的点, 然后在下一层进行搜索时, 从上一层离查询最近的点开始搜索。当搜索到最底层时, 再使用图2.5所示的算法进行搜索, 该算法通过维护两个优先队列 pq 和 H, 优先队列 pq 表示候选队列, H 表示大小为 ef 的结果队列。在每次迭代中, 选择 pq 中距离查询最近的点 v_c , 并探索点 v_c 的所有邻居。当算法找不到距离查询更近的点时, 便停止搜索。

尽管 HNSW 采用了很符合直觉的层次结构来加速搜索, 但是目前最新

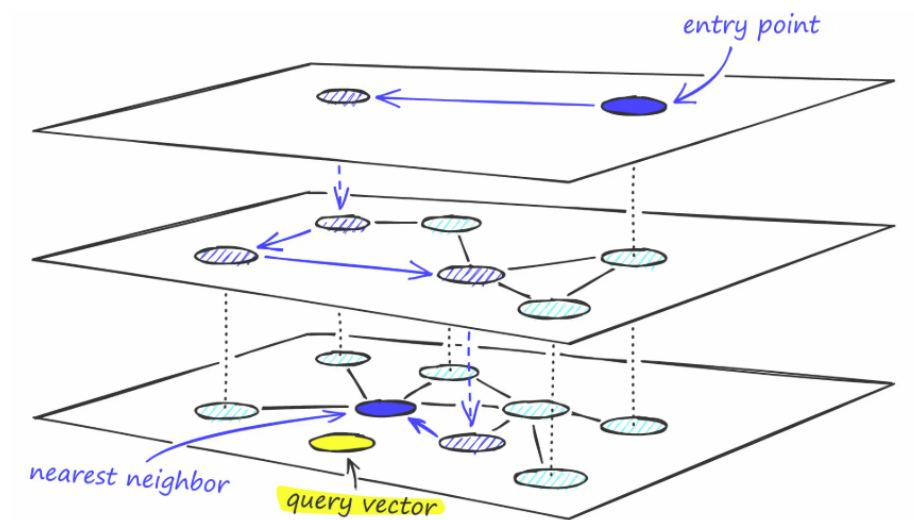


图 2.4: HNSW 查询过程

Algorithm 1 Greedy search (graph G , query q , entry point ep , ef)

```

1:  $pq$  = a priority queue with unlimited capacity, initialized with  $ep$ 
2:  $H$  = a max-heap with capacity  $ef$ 
3: while  $pq$  is not empty do
4:    $d_{v_c}, v_c$  = pop an element from  $pq$ 
5:    $d_{v_{top}}, v_{top}$  = the heap top of  $H$ 
6:   if  $d_{v_c} > d_{v_{top}}$  then
7:     break
8:   for each neighbor  $v$  of  $v_c$  which has not been accessed do
9:     if  $D(v, q) < d_{v_{top}}$  then
10:      Insert  $(D(v, q), v)$  into  $pq$  and  $H$ 
11:      mark  $v$  as accessed
12:      resize  $H$  to be  $ef$ 
13: return  $k$  smallest elements in  $H$ 

```

图 2.5: HNSW 查询过程伪代码

的研究表明 HNSW 的层次结构在高维空间中作用有限¹。所以同学们可以只使用 HNSW 的底层图并随机选择入口点，并且可以只关注底层图搜索算法的并行化。

¹实际上，HNSW 真正有效的部分在于底层图构建时使用三角不等式进行裁边，但是在论文中仅仅一笔代过，而没有给出详细讨论。如果对图索引构建理论感兴趣的同学可以参考论文Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph

对于图索引上查询内的并行化，实际上是一个非常困难的问题，目前学术界也只有很少的解决方案²。但是同学们仍然可以简单探索一些方法，例如（1）选题指导书中提到的边划分并行、Layer 0 点划分并行、多入口点并行；（2）采用 IVF+HNSW 的嵌套结构，建立多个 HNSW，每个线程负责搜索一部分 HNSW。上述的优化很可能导致负优化，同学们只要如实汇报实验结果并给出一定分析即可。

3 口令猜测选题：多线程实验

3.1 口令猜测过程的并行化

PCFG 算法的具体原理，以及其并行化过程，请参照学期初发给大家的“口令猜测算法并行化选题.pdf”，此处不再赘述。这里主要结合框架代码介绍大家需要进行并行化的内容。

如图3.6、3.7所示，这里的两个循环，在本质上是给一个 PT 的最后一个 segment 进行具体的填充。例如，对于 L_8D_2 这个 PT 而言，在先前优先队列的初始化和不断迭代过程中，已经将其中的 L_8 进行实例化了，只需要在这里将 D_2 进行填充即可。这时，假设模型统计到了 12、11、23 这三个具体的值，那么这个循环就会逐一将这三个值填充到 D_2 里面，形成三个新的口令。

显而易见的是，这个过程是可以并行化的。你需要通过 pthread/OpenMP 将这个循环进行并行化，同时改变头文件、main 函数等文件，使不同线程的返回结果能够存起来，并且按照 main 函数里面的方式进行哈希和清理。

和 SIMD 作业中哈希值的正确性不同，口令猜测很难直接检测“正确性”，实际操作中往往用相当长一段时间内的口令攻破成功率来评估口令猜测的并行算法。但是，由于服务器的计算资源相对有限，暂时不能支持千万级别的口令结果存储，所以你可以采用一个简单的方法来测试你并行生成的口令是否正确。你可以将飞书群里面的 correctness_guess.cpp 文件上传到服务器的 guess 目录下，将这个文件代替 main 函数进行编译。编译并运行之后，你会发现输出的信息多了一行“Cracked”，这就是猜测算法的攻破率了。

²感兴趣的同学可以阅读论文iQAN: Fast and Accurate Vector Search with Efficient Intra-Query Parallelism on Multi-Core Architectures, 但由于论文难度较大，这里并不要求同学们复现该论文，如果阅读后理解了该论文可以尝试把论文中的一些方法用到自己的并行实现中。